

Project Executable Files

Date	01 July 2026
Team ID	
Project Name	Rising Waters-ML Flood Prediction System
Maximum Marks	3 Marks

Project Executable Files

Step 1:Submission Checklist

S.No	Item to Submit	Submitted (Yes/No/NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File/Folder Structure

Flood-Prediction-System/

|

|— app.py

|— Procfile

|— requirements.txt

|— README.md

|— .gitignore

|

|— Dataset/

| |— Flood_Prediction_Final.xlsx

|

|— models/

| |— floods.save

| |— transform.save

|

|— Notebook/

| |— Flood_Prediction.ipynb

|

|— reports/

| |— Flood_Report.pdf

|

|— Static/

| |— css/

```
| |— js/
| |— images/
|
|— Templates/
| |— home.html
| |— index.html
| |— chance.html
| |— no_chance.html
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://rising-waters-ml-flood-prediction-system.onrender.com
Login Credentials (Demo)	Not Required
Platform / Hosting Provider	Local Flask Server (IBM Cloud - Future Deployment)
Repository Link	https://github.com/chandeeswarimothukoori-com/mits/Flood-Prediction-System
Demo Video Link	https://drive.google.com/file/d/1Oh6DDmfSZ0e0S9R2NfIMi3daJsoAN97Q/view?usp=drivesdk

Step 4: Run Instructions

1. Install Python 3.8 or above.
2. Open the project folder in Visual Studio Code.
3. Install the required libraries using:
4. `pip install -r requirements.txt`
5. Ensure the trained model files (floods.save and transform.save) are present inside the models folder.
6. Open the terminal in the project folder.
7. Run the Flask application using:

8. python app.py
9. The Flask server starts running on the local system.
10. Open the browser and visit:
11. `http://127.0.0.1:5000`
12. Enter the required weather and rainfall parameters.
13. View the generated flood prediction result.

Deployment using Render

14. Push the project code to the GitHub repository.
15. Create an account on Render and connect the GitHub account.
16. Create a new Web Service and select the project repository.
17. Configure the deployment settings:
18. Runtime: Python 3
19. Branch: main
20. Build Command:

```
pip install -r requirements.txt
```
21. Start Command:

```
gunicorn app:app
```
22. Deploy the application on Render.
23. After successful deployment, obtain the generated Render URL.
24. Open the deployed URL in the browser and verify that the application is working.
25. Enter the required weather parameters and test the flood prediction functionality.
26. Render automatically redeploys the application whenever new changes are pushed to the connected GitHub repository.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	The application currently works only with manually entered weather data.	Future enhancement: Integrate a real-time weather API.
2	The project is not yet deployed to IBM Cloud.	Deployment is planned as a future enhancement.
3	No user authentication or prediction history is implemented.	Can be added using a database and login system in future versions.